

Règles d'association

Présentation du problème

- De larges volumes de données sont stockés par des transactions journalières
- Ex: dans les grandes surfaces, les achats des consommateurs sont enregistrés tous les jours
- Les commerçants s'intéressent à l'analyse des données pour mieux appréhender les besoins de leurs clients
- **Problème posé**
 - Processus de découverte de motifs fréquents ou d'associations entre les données de bases de données

Présentation du problème

- Analyse du panier de la ménagère
 - Quels sont les ensembles d'articles qui sont souvent achetés ensemble?
 - Ex : Pain, Lait $\Rightarrow$ Beurre [10%,90%]
- Application à la gestion de stocks, au marketing
- Analyse de relations ou corrélations entre attributs d'une base de données relationnelle
 - Client(x,Avocat,Dakar), Ventes(x,y) $\Rightarrow$ Produit(y,Lait) [15%,75%]
- Beaucoup de domaines d'application : médecine (diagnostic), analyse de données spatiales,...

Extraction de règles d'association

- Formalisation du problème
 - Données du problème
 - Un ensemble d'articles (items) $I = \{ i_1, i_2, \dots, i_n \}$
 - Un ensemble de transactions D , chaque transaction T_j de D étant un sous-ensemble de I .
 - Exemple :
 - $I = \{ \text{Pain}, \text{Lait}, \text{Beurre}, \text{Sucre} \}$
 - $D = \{ T_1, T_2, T_3, T_4, T_5 \}$
 - $T_1 = \{ \text{Pain}, \text{Lait}, \text{Beurre} \}$
 - $T_2 = \{ \text{Pain}, \text{Lait}, \text{Sucre} \}$
 - $T_3 = \{ \text{Pain} \}$
 - $T_4 = \{ \text{Pain}, \text{Lait} \}$
 - $T_5 = \{ \text{Pain}, \text{Lait}, \text{Beurre}, \text{Sucre} \}$

Extraction de règles d'association

- Formalisation du problème
 - Motifs recherchés
 - Règles d'associations intéressantes
 - Règle d'association : $X \Rightarrow Y$, où X et Y sont des sous-ensembles de I , $X \cap Y = \emptyset$
 - *Pain, Lait $\Rightarrow$ Sucre*
 - Deux mesures de base
 - Support d'une règle d'association
 - Confiance d'une règle d'association

Support d'une règle d'association

- **Définition**
- Un ensemble d'items est appelé un **itemset**.
- Le support d'un itemset est le nombre de transactions qui contiennent cet itemset.
- La règle d'association $X \Rightarrow Y$ a un **support** de s dans la base D si $s\%$ des transactions T de D contiennent $X \cup Y$.

$$\begin{aligned} \text{Sup}(X \cup Y / D) &= | \{ T \in D / T \supseteq X \cup Y \} | / | D | \\ &= P(X \cup Y) \end{aligned}$$

- **Exemple**

- $T_1 = \{ \text{Pain}, \text{Lait}, \text{Beurre} \}$

$$T_4 = \{ \text{Pain}, \text{Lait} \}$$

- $T_2 = \{ \text{Pain}, \text{Lait}, \text{Sucre} \}$

$$T_5 = \{ \text{Pain}, \text{Lait}, \text{Beurre}, \text{Sucre} \}$$

- $T_3 = \{ \text{Pain} \}$

$$\text{Sup}(\text{Pain}, \text{Beurre}) = 2/5$$

$$\text{Sup}(\text{Pain}, \text{Lait}, \text{Sucre}) = 2/5$$

Confiance d'une règle d'association

- Définition

- La règle d'association $X \Rightarrow Y$ a une **confiance** de c dans la base D si $c\%$ des transactions T de D contenant X contiennent Y .

$$Conf(X \Rightarrow Y / D) = Sup(X \cup Y / D) / Sup(X / D)$$

- Exemple

- $T_1 = \{ \text{Pain, Lait, Beurre} \}$

$$T_4 = \{ \text{Pain, Lait} \}$$

- $T_2 = \{ \text{Pain, Lait, Sucre} \}$

$$T_5 = \{ \text{Pain, Lait, Beurre, Sucre} \}$$

- $T_3 = \{ \text{Pain} \}$

$$Conf(\text{Pain} \Rightarrow \text{Beurre}) = 2/5$$

$$Conf(\text{Pain, Lait} \Rightarrow \text{Sucre}) = 2/4$$

Règles intéressantes

- Définition
 - Étant donné deux seuils fixés a priori par un utilisateur
 - Un seuil minimal de support « minsup » et
 - Un seuil minimal de confiance « minconf »
 - Une règle $X \Rightarrow Y$ est **intéressante** dans D si
 - $Sup(X \cup Y / D) \geq minsup$ et $Conf(X \Rightarrow Y / D) \geq minconf$
- Problème posé
 - Étant donné un ensemble de transactions D et deux seuils minimaux « minsup » et « minconf »
 - Déterminer toutes les règles d'association intéressantes

Règles intéressantes

- Exemple

- $T_1 = \{ \text{Pain, Lait, Beurre} \}$

$$T_4 = \{ \text{Pain, Lait} \}$$

- $T_2 = \{ \text{Pain, Lait, Sucre} \}$

$$T_5 = \{ \text{Pain, Lait, Beurre, Sucre} \}$$

- $T_3 = \{ \text{Pain} \}$

- Etant donné $\text{minsup} = 2/5$, $\text{minconf} = 3/4$

- $\text{Lait, Beurre, Sucre} \Rightarrow \text{Pain}$ (non intéressante)

- $\text{Conf} = 1/1 \geq \text{minconf}$ mais $\text{Sup} = 1/5 \leq \text{minsup}$

- $\text{Pain, Lait} \Rightarrow \text{Beurre}$ (non intéressante)

- $\text{Sup} = 2/5 \geq \text{minsup}$ mais $\text{Conf} = 2/4 \leq \text{minconf}$

- $\text{Lait, Beurre} \Rightarrow \text{Pain}$ (intéressante)

- $\text{Sup} = 2/5 \geq \text{minsup}$ et $\text{Conf} = 1 \geq \text{minconf}$

Algorithme d'extraction

- Algorithme de base
 - Algorithme « A-priori » introduit par R. Agrawal en 1994
 - Voir « <http://www.almaden.ibm.com/u/ragrawal> »
 - Deux phases principales
 1. Calcul de tous les itemsets fréquents, i.e. des ensembles d'articles X tels que $Sup(X / D) \geq minsup$
 2. Calcul de toutes les règles d'association de la forme $X / Y \Rightarrow Y$ intéressantes
 - Pour tout itemset fréquent X
 - Pour tout sous-ensemble A de X non vide, soit $B = X - A$
 - Rechercher toutes les règles intéressantes de la forme $A \Rightarrow B$
- La phase (1) est la phase la plus coûteuse

Calcul des itemsets fréquents

- Principe de base
 - Basé sur la propriété fondamentale du support (anti-monotonie)
 - Si $X \subseteq Y$, alors $Sup(X / D) \geq Sup(Y / D)$ pour tout D
 - Tout sous-ensemble d'un itemset fréquent est fréquent
 - Tout sur-ensemble d'un itemset non fréquent est non fréquent

Proposition : treillis des itemsets

L'ensemble des itemsets peut être partiellement ordonné sous forme d'un treillis par inclusion ensembliste

Calcul des itemsets fréquents

➤ Recherche ascendante des itemsets fréquents

- Etape 1 : calcul des 1-itemsets fréquents = sous-ensembles fréquents de I comportant 1 article
- Etape k : calcul des k -itemsets fréquents (sous-ensemble de I comportant k articles) à partir des $(k-1)$ itemsets fréquents

Calcul des itemsets fréquents

- Algorithme « A-priori »
 - **Entrée** : un ensemble D de transactions T_i , minsup
 - **Sortie** : ensemble L de tous les itemsets fréquents
 - **Méthode**
 - Calculer l'ensemble $L(1)$ des 1-itemsets fréquents
 - $K = 2$
 - Tant que $L(K-1)$ n'est pas vide
 - // **Phase de génération des candidats**
 - Calculer l'ensemble $C(K)$ des K -itemsets candidats à partir de $L(K-1)$
 - // **Phase d'évaluation des candidats**
 - Calculer le support de chaque candidat dans $C(K)$
 - Soit $L(K)$ l'ensemble des candidats C de $C(K)$ tels que $\text{Sup}(C/D) \geq \text{minsup}$
 - $L(K)$ est l'ensemble de tous les K -itemsets fréquents
 - $K=K+1$
 - Retourner l'union de tous les ensembles $L(K)$

Génération des candidats

- Principe de base
 - **Entrée** : ensemble $L(K-1)$ des $K-1$ itemsets fréquents
 - **Sortie** : ensemble $C(K)$ des K -itemsets candidats
 - **Méthode**
 - // **Phase de jointure ou composition**
 - Soit $C(K)$ l'ensemble des K -itemsets C tels que
 - $C = F1 \cup F2$ où $F1$ et $F2$ sont éléments de $L(K-1)$
 - $F1 \cap F2$ comporte $(K-2)$ éléments
 - // **Phase d'élagage**
 - Supprimer de $C(K)$ tout candidat C tel qu'il existe
 - Un sous-ensemble de C de $(K-1)$ éléments non présent dans $L(K-1)$
 - Retourner $C(K)$
 - Note : la génération des candidats se fait en mémoire, sans accéder aux données

Exemple de génération de candidats

- $L_3 = \{ abc, abd, acd, ace, bcd \}$
- Jointure : $L_3 * L_3$
 - abcd à partir de abc et abd
 - acde à partir de acd et ace
- Elagage
 - acde est élagué puisque ade n'est pas dans L_3
- $C_4 = \{ abcd \}$

Evaluation des candidats

- Principe de base
 - **Entrée** : ensemble $C(K)$ de K -itemsets candidats, ensemble D des transactions, minsup
 - **Sortie** : ensemble $L(K)$ des K -itemsets fréquents
 - **Méthode naïve**
 - Initialiser $L(K)$ à l'ensemble vide
 - Pour tout candidat C de $C(K)$, initialiser une variable $C.Sup$ à 0
 - Pour toute transaction T_j de D
 - Pour tout candidat C de $C(K)$
 - » Si C est inclus dans T_j , alors $C.Sup++$
 - Pour tout candidat C de $C(K)$
 - Si $C.Sup \geq \text{minsup}$, alors Ajouter C à $L(K)$
 - Retourner $L(K)$
 - Note : l'évaluation des candidats se fait en une seule passe sur les données

Exemple simple

Calcul de $L(1)$

$T1 = \{\text{Pain, Lait, Beurre}\}$

$T2 = \{\text{Pain, Lait, Sucre}\}$

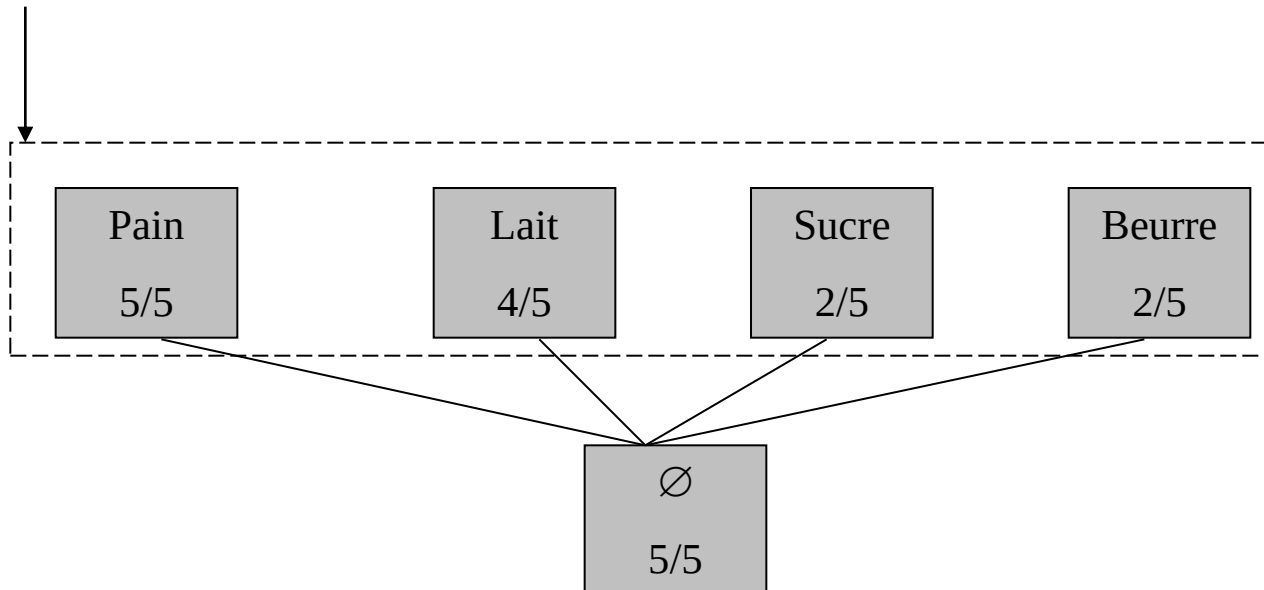
$T3 = \{\text{Pain}\}$

$T4 = \{\text{Pain, Lait}\}$

$T5 = \{\text{Pain, Lait, Beurre, Sucre}\}$

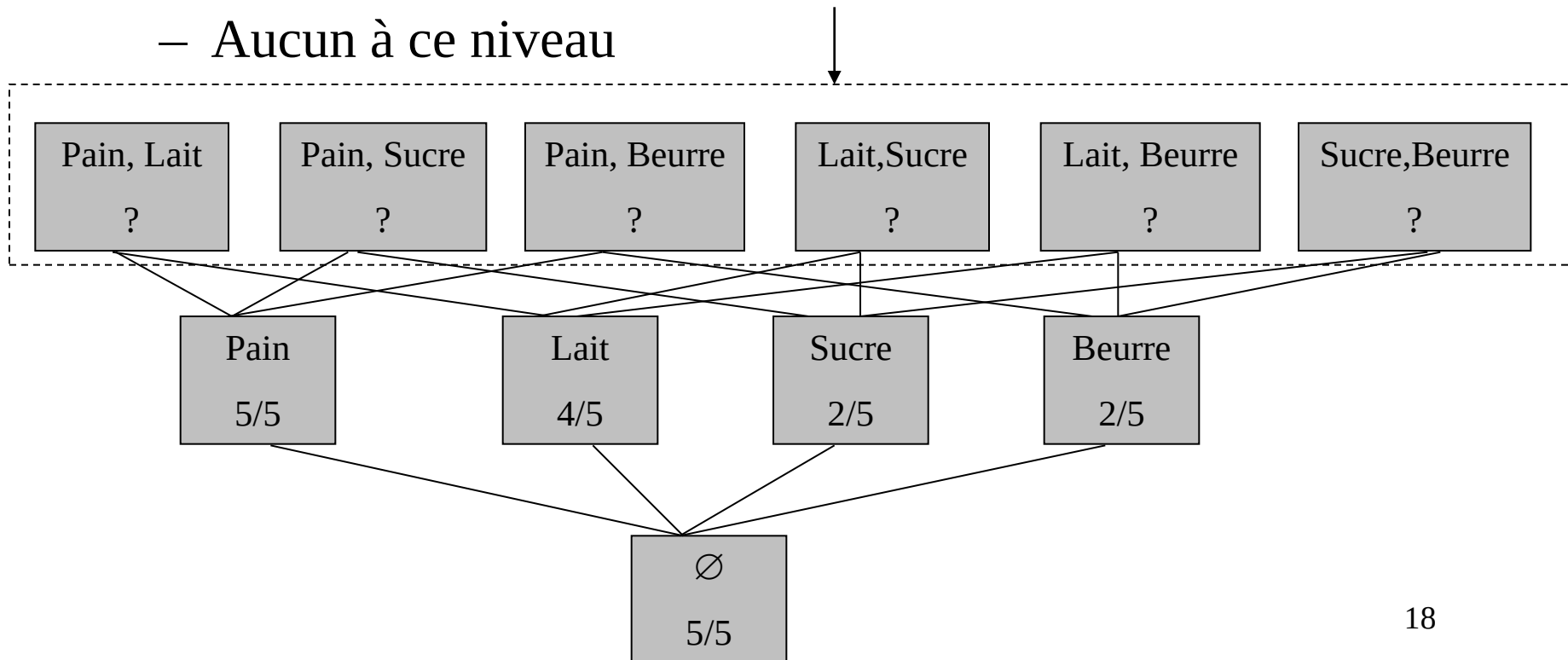
$\text{minsup} = 2/5$

$L(1)$



Génération de C(2)

- Composition des éléments de L(1)
 - Union deux à deux
- Elagage des éléments de C(2)
 - Aucun à ce niveau



Evaluation de C(2)

T1 = {Pain, Lait, Beurre}

T2 = {Pain, Lait, Sucre}

T3 = {Pain}

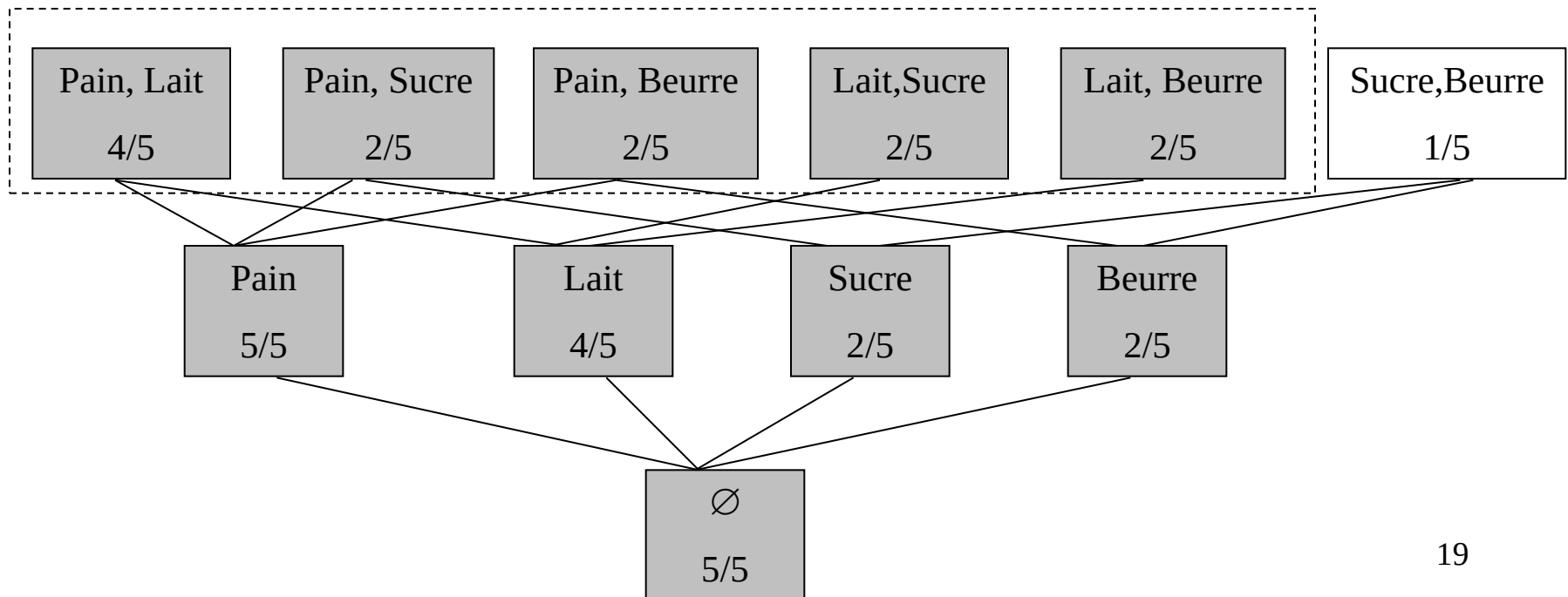
T4 = {Pain, Lait}

T5 = {Pain, Lait, Beurre, Sucre}

minsup = 2/5

- Calcul de L(2)

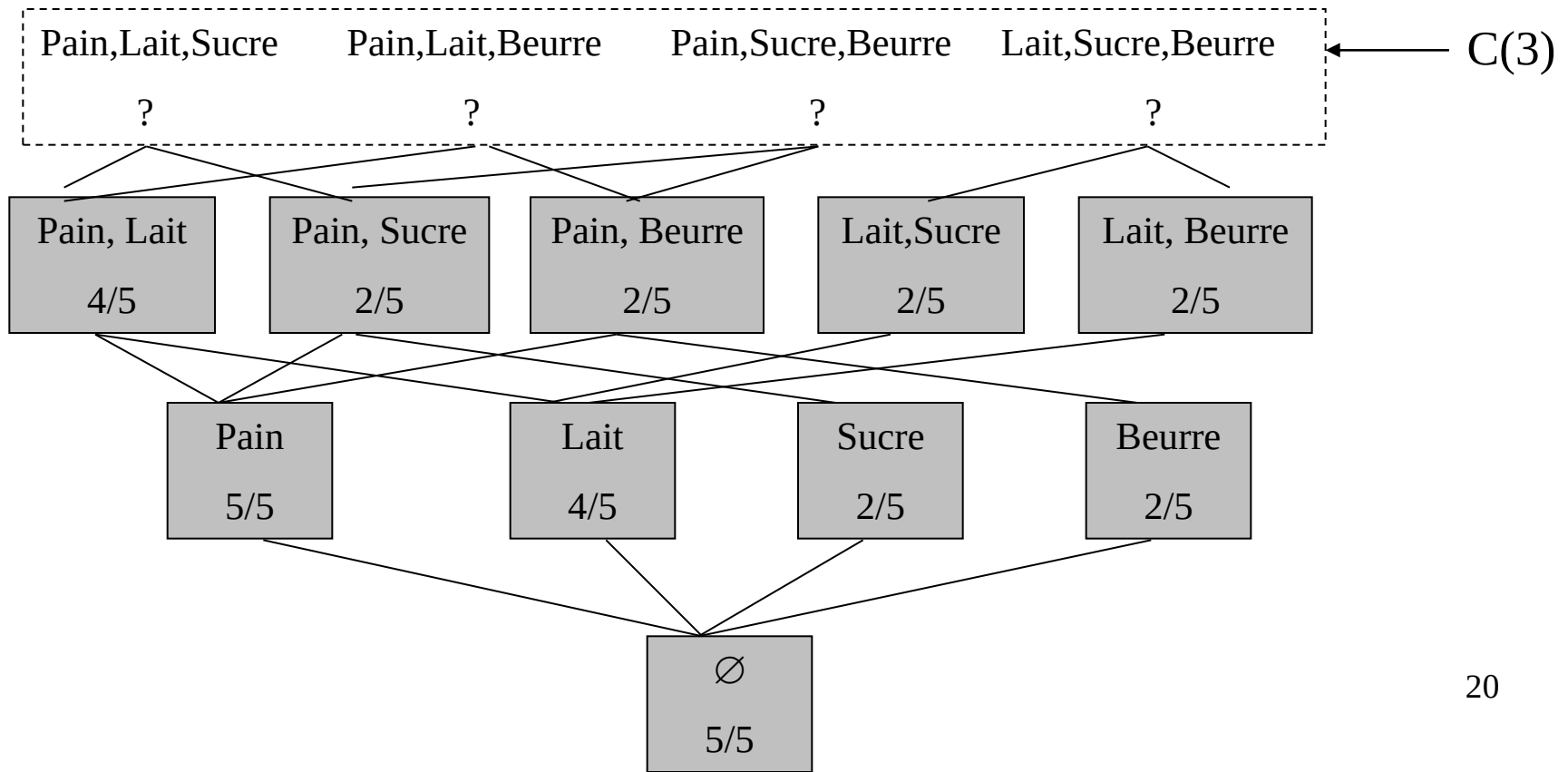
En une passe sur les données



Génération de C(3)

- Composition des éléments de L(2)

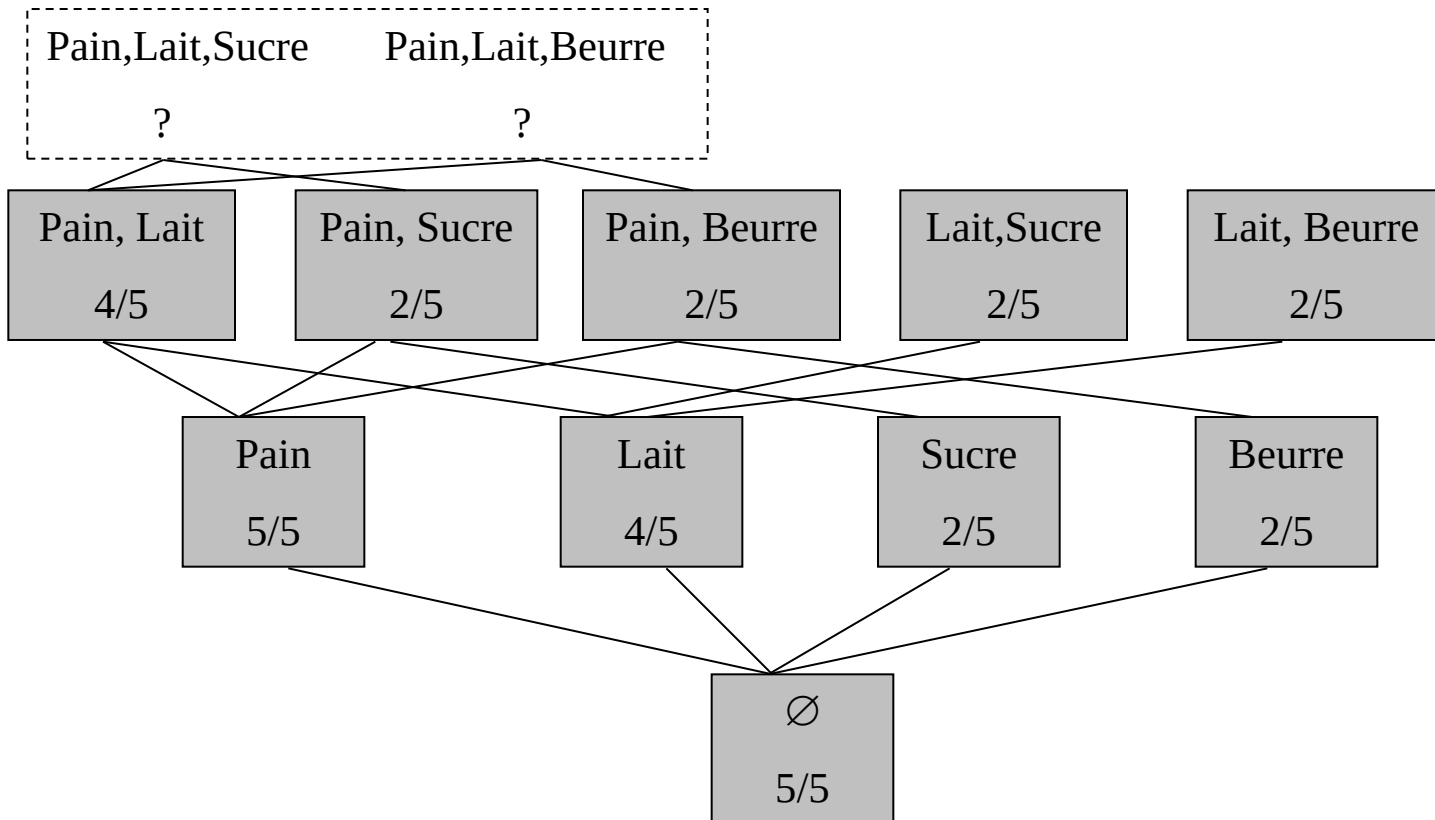
En combinant seulement les itemsets dont les premiers éléments sont identiques



Génération de C(3)

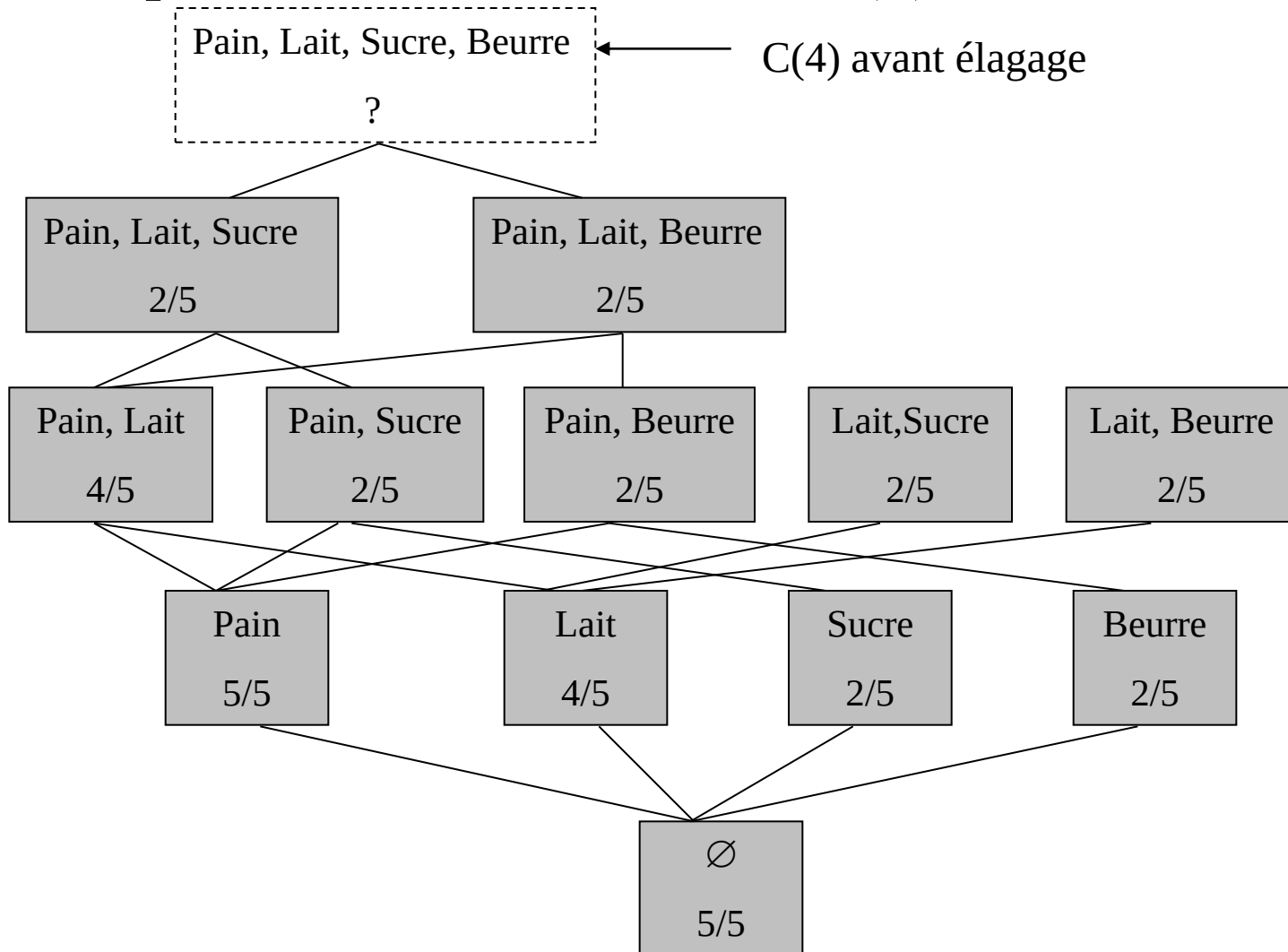
- Elagage des éléments de C(3)

Suppression de C(3) des candidats [Pain, Sucre, Beurre] et [Lait, Sucre, Beurre] car [Sucre, Beurre] non présent dans L(2)



Génération de C(4)

- Composition des éléments de L(3)

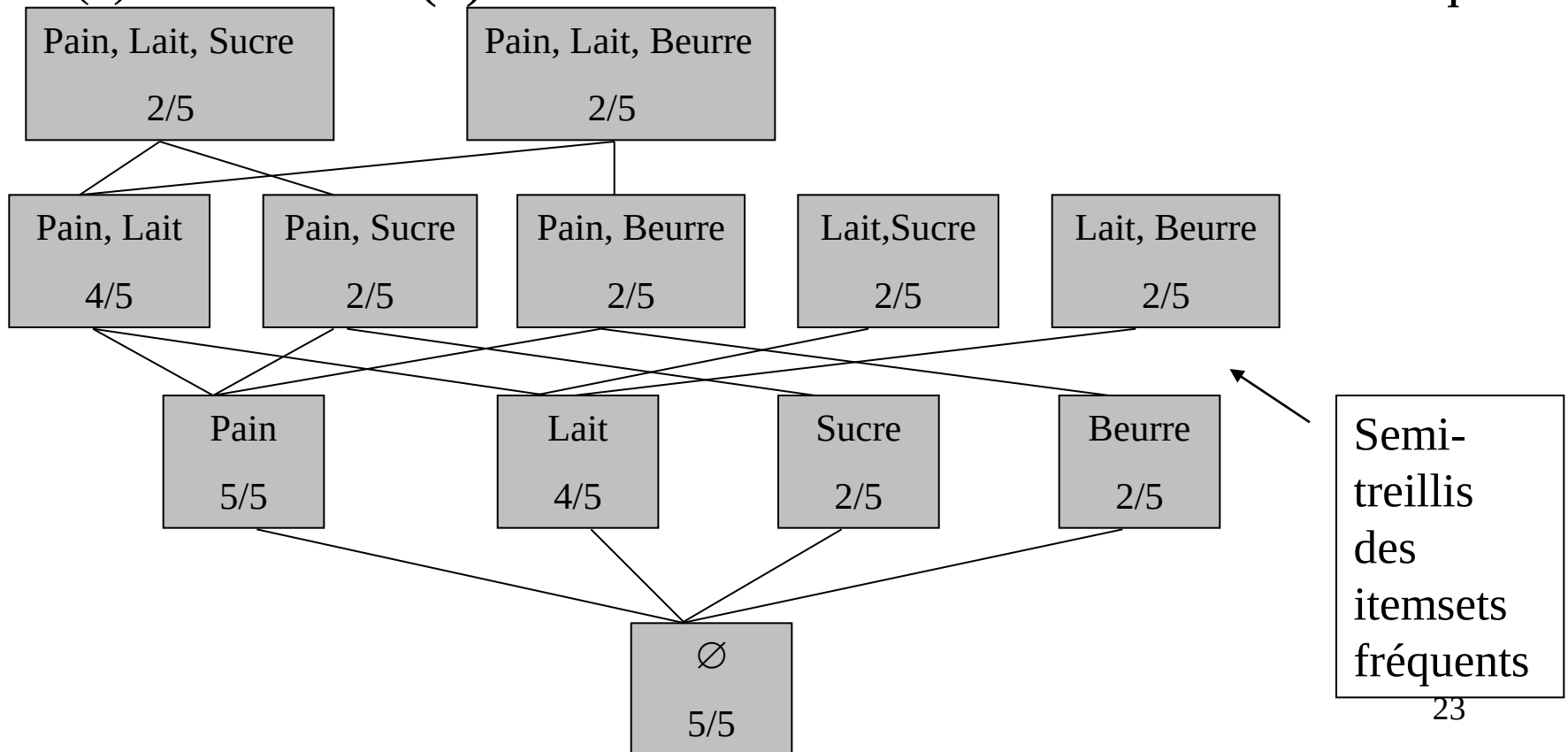


Génération de C(4)

- Elagage des éléments de C(4)

- Suppression de [Pain, Lait, Sucre, Beurre] car [Lait, Sucre, Beurre] non présent dans L(3)

- C(4) est vide $\Rightarrow$ L(4) est vide $\Rightarrow$ Fin du calcul des itemsets fréquents



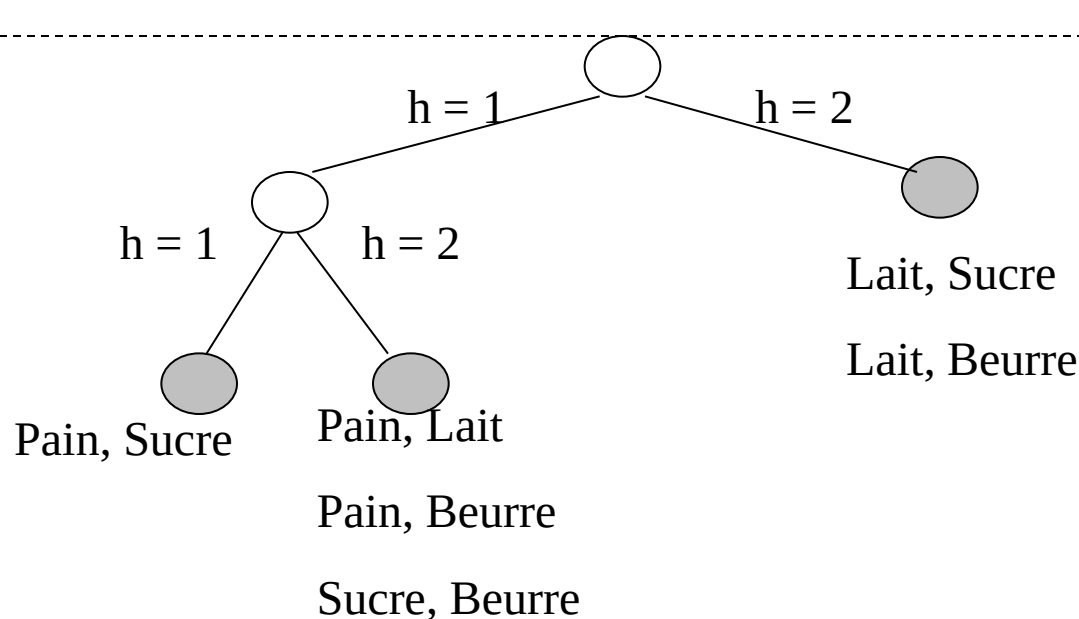
Calcul des règles intéressantes

- Méthode naïve
 - Pour tout K-itemset X fréquent (avec $K > 1$) et $Y \subset X$
 - Calculer la confiance de la règle d'association $X / Y \Rightarrow Y$
 - Si elle est supérieure à minconf, la règle générée est intéressante
 - Note : cette phase ne nécessite aucun accès aux données
- Suite exemple
 - Avec minconf = 3/4
 - $X = \{\text{Pain, Lait, Beurre}\}$ est fréquent
 - Pain, Lait $\Rightarrow$ Beurre non intéressante car $\text{Conf} = 2/4$
 - Pain, Beurre $\Rightarrow$ Lait intéressante car $\text{Conf} = 2/2$
 - Lait, Beurre $\Rightarrow$ Pain intéressante car $\text{Conf} = 2/2$
 - Pain $\Rightarrow$ Lait, Beurre non intéressante car $\text{Conf} = 2/5$
 - Lait $\Rightarrow$ Pain, Beurre non intéressante car $\text{Conf} = 2/4$
 - Beurre $\Rightarrow$ Pain, Lait non intéressante car $\text{Conf} = 2/4$

Optimisation classique

- Elagage et évaluation des candidats de niveau K
 - Stockage des candidats de niveau de K dans un arbre N-aire de hachage
 - Fonction de hachage portant sur les articles des candidats
 - Exemple simple
 - $h(\text{Pain}) = 1$, $h(\text{Lait}) = 2$, $h(\text{Sucre}) = 1$, $h(\text{Beurre}) = 2$

C(2) → Pain, Lait Pain, Sucre Pain, Beurre, Lait, Sucre Lait, Beurre Sucre, Beurre

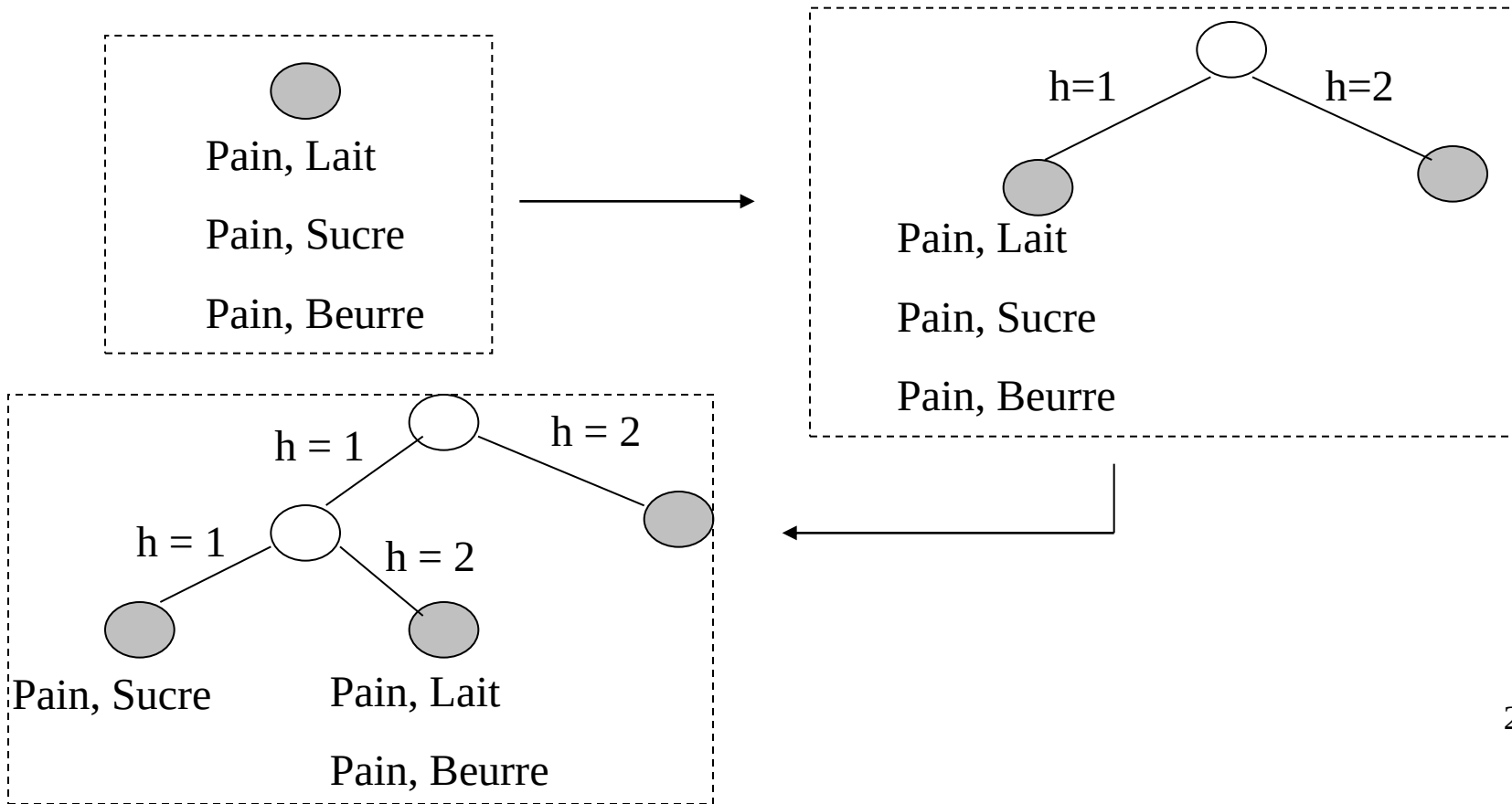


Arbre de hachage

- Méthode de construction
 - Au départ, une racine sans feuilles ni candidats
 - A chaque étape, on ajoute un candidat à l'arbre existant
 - Recherche de la feuille où devrait se trouver le candidat
 - Ajout du candidat à cette feuille
 - Si une feuille est trop pleine après ajout (plus de N candidats)
 - On la scinde en P , où P est le nombre de valeurs possibles de la fonction de hachage
 - On répartit les candidats de l'ancienne feuille dans les P nouvelles feuilles

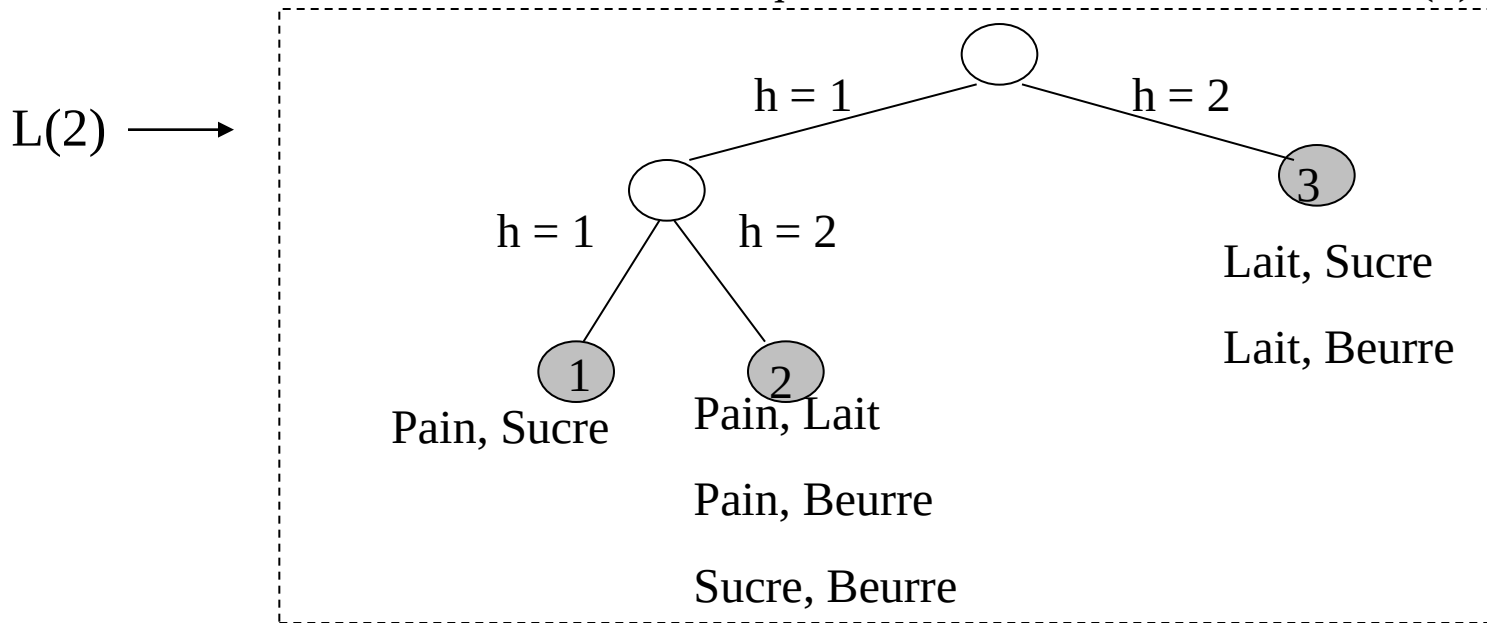
Arbre de hachage

- Exemple de construction
 - $N=2$, $P=2$, $h(\text{Pain})=1$, $h(\text{Lait})=2$, $h(\text{Sucre})=1$, $h(\text{Beurre})=2$
 - Ajout de $C1 = \{\text{Pain}, \text{Lait}\}$, $C2 = \{\text{Pain}, \text{Sucre}\}$ puis $C3 = \{\text{Pain}, \text{Beurre}\}$



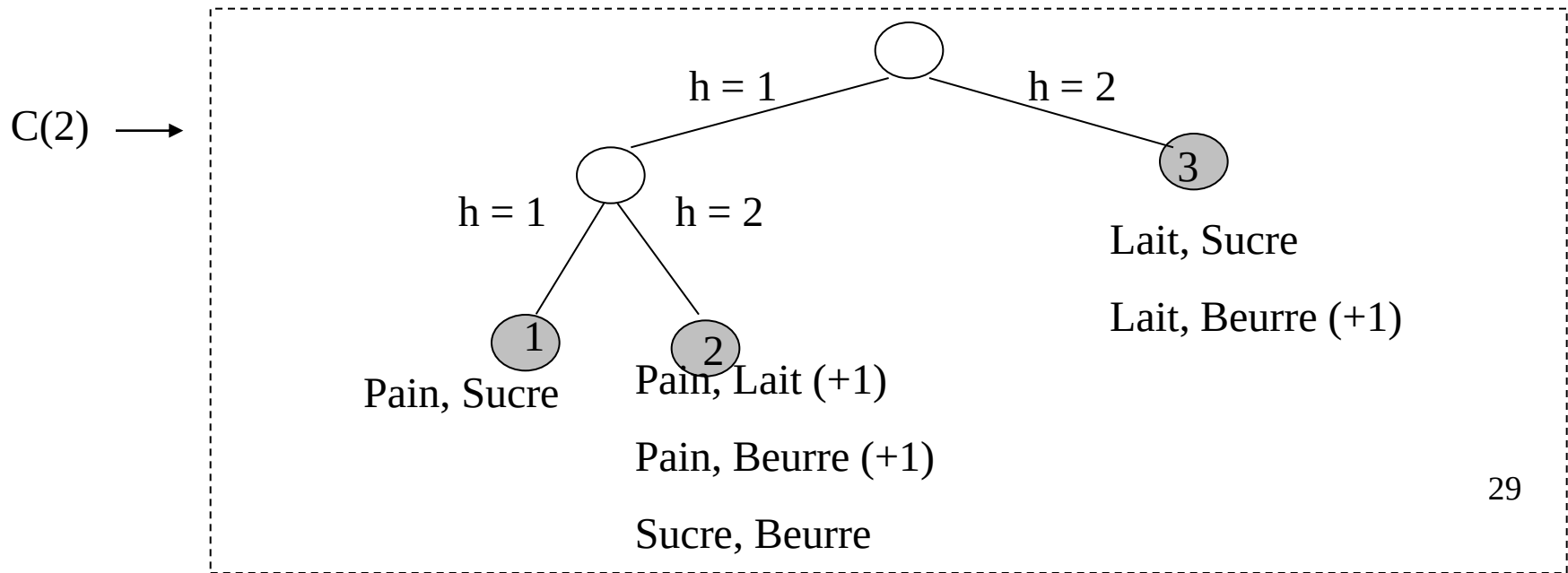
Arbre de hachage

- Utilisation lors de la phase d'élagage
 - $h(\text{Pain}) = 1$, $h(\text{Lait}) = 2$, $h(\text{Sucre}) = 1$, $h(\text{Beurre}) = 2$
 - Exemple : tous les sous-ensembles de [Pain, Sucre, Beurre] sont-ils éléments de $L(2)$?
 - Recherche successive
 - [Pain, Sucre] uniquement recherché dans la feuille (1)
 - [Pain, Beurre] uniquement recherché dans la feuille (2)
 - [Sucre, Beurre] uniquement recherché dans la feuille (2)



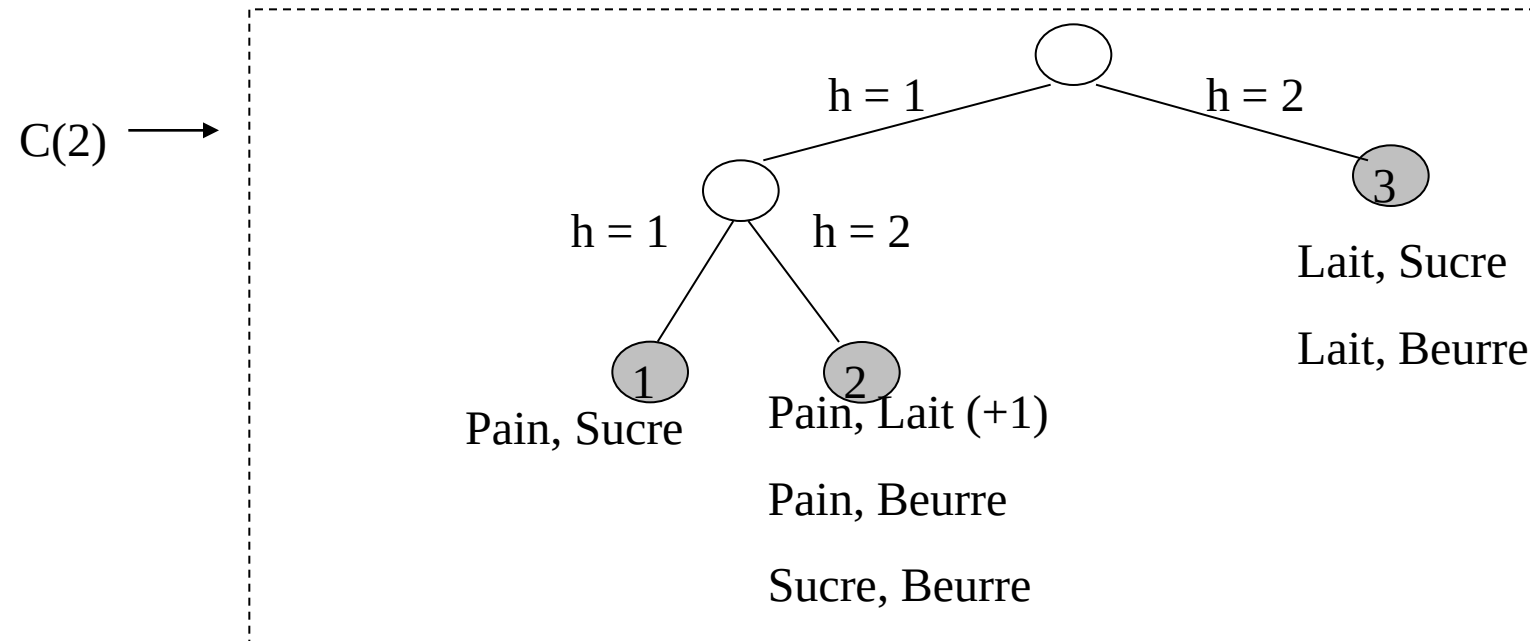
Arbre de hachage

- Utilisation lors de la phase d'évaluation
 - $h(\text{Pain}) = 1$, $h(\text{Lait}) = 2$, $h(\text{Sucre}) = 1$, $h(\text{Beurre}) = 2$
 - Exemple : quels candidats de $C(2)$ sont inclus dans $T1 = \{\text{Pain, Lait, Beurre}\}$
 - Recherche récursive des sous-ensembles de $T1$ de deux éléments
 - $\{\text{Pain, Lait}\} \Rightarrow$ évaluation des candidats de la feuille (2)
 - $\{\text{Pain, Beurre}\} \Rightarrow$ évaluation des candidats de la feuille (2) déjà effectuée
 - $\{\text{Lait, Beurre}\} \Rightarrow$ évaluation des candidats de la feuille (3)
 - Les candidats de la feuille (1) ne sont pas évalués pour $T1$



Arbre de hachage

- Utilisation lors de la phase d'évaluation
 - $h(\text{Pain}) = 1$, $h(\text{Lait}) = 2$, $h(\text{Sucre}) = 1$, $h(\text{Beurre}) = 2$
 - Exemple : quels candidats de $C(2)$ sont inclus dans $T4 = \{\text{Pain}, \text{Lait}\}$
 - Recherche récursive des sous-ensembles de $T4$ de deux éléments
 - Seulement $\{\text{Pain}, \text{Lait}\} \Rightarrow$ évaluation des candidats de la feuille (2)
 - Les candidats des feuilles (1) et (3) ne sont pas évalués pour $T4$



Optimisations de A-priori

- Par la réduction du nombre de passes dans la base
 - Technique de hachage, DHP(Direct Hashing and Pruning), Park et al
 - Lors du parcours des données pour la génération de l'ensemble L_1 des 1-itemsets fréquents à partir de l'ensemble C_1 , tous les 2-itemsets sont générés pour chaque transaction
 - Ensuite, une fonction de hachage est utilisée pour les répartir dans différents buckets d'une table de hachage
 - Chaque bucket a un compteur incrémenté à chaque insertion d'un itemset
 - Un 2-itemset qui se trouve dans un bucket dont le compteur est inférieur au seuil de support ne peut pas être fréquent

Optimisations de A-priori

Adresse des buckets	0	1	2	3	4	5	6
Compteur des buckets	2	1	2	3	2	1	0
Contenu des buckets	AD AD	AE	AF BC	BD DE DE	DF BE	AB	

TID Liste items

T1 A D F

T2 A B D E

T3 B C

T4 F

T5 D E

$$h(x,y) = ((\text{order of } x) * 10 + (\text{order of } y)) \bmod 7$$

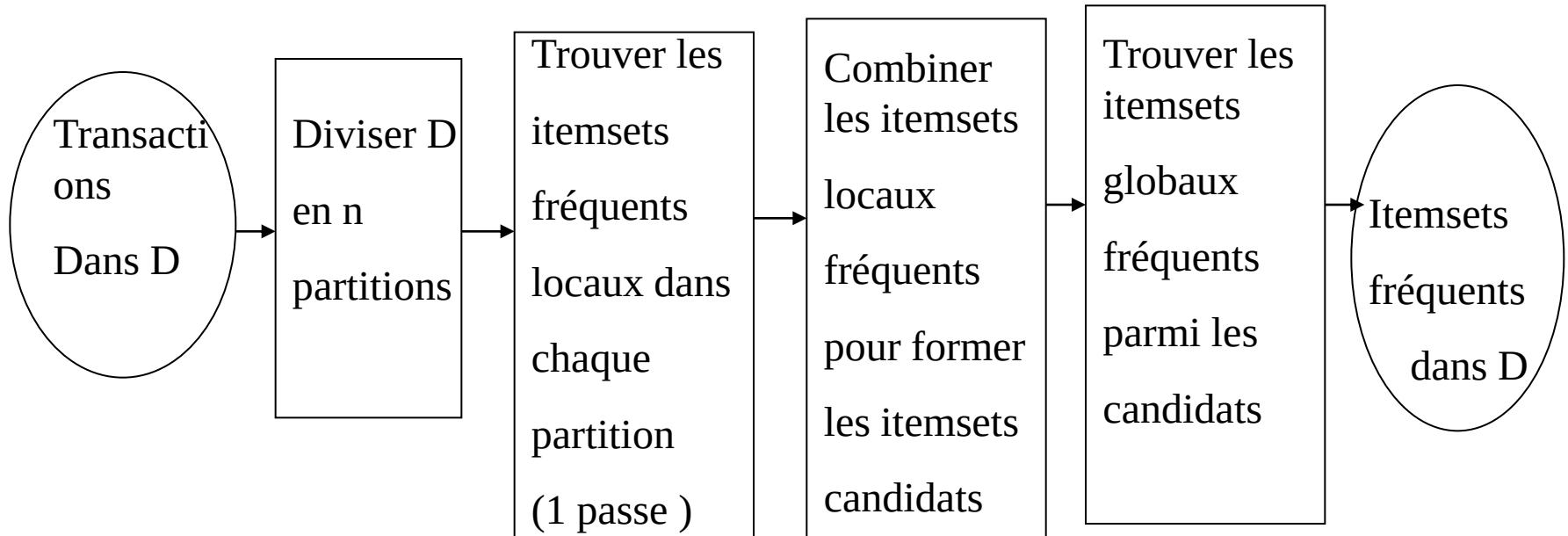
– Réduction du nombre de transactions

- Une transaction ne contenant aucun K-itemset ne contiendra aucun (K+1) itemset

– Partitionnement de l'ensemble D des transactions en N ensembles D_i

- Introduit par A. Savasere et al
- Un itemset doit être fréquent dans au moins un D_i pour être fréquent sur D

Optimisations de A-priori



Deux passes dans la base de données

Passé 1 : partitionnement de la base et détermination des itemsets fréquents locaux

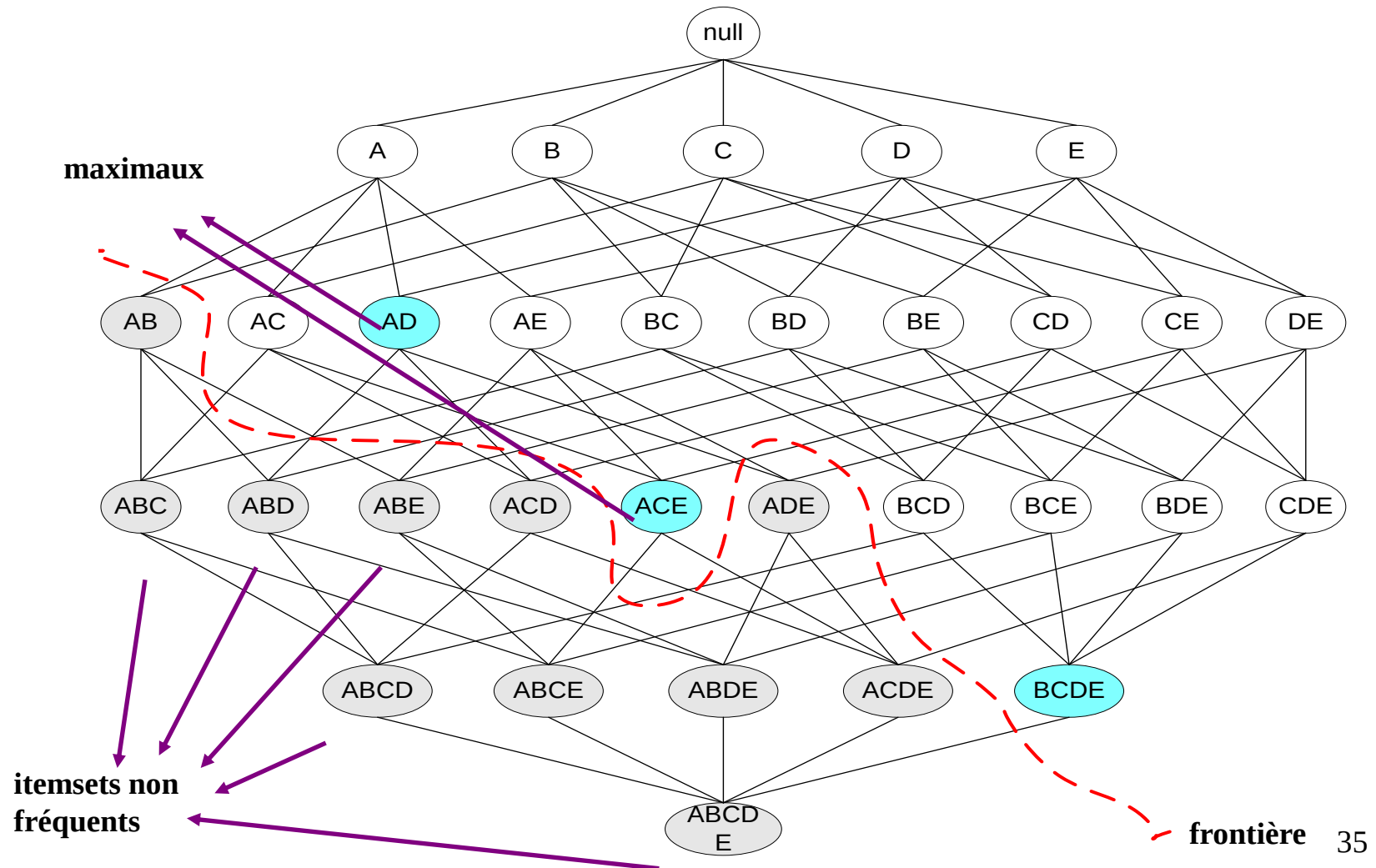
Passé 2 : détermination des itemsets fréquents globaux

Optimisations de A-priori

- Par la réduction du nombre de fréquents générés
 - Idée générale : trouver une partie de l'ensemble des fréquents et en déduire les autres
 - Les maximaux
 - Un itemset fréquent est maximal si aucun de ses sur-ensembles immédiats n'est fréquent
 - Les fermés
 - Un itemset est fermé si aucun de ses sur-ensembles immédiats n'a le même support que lui

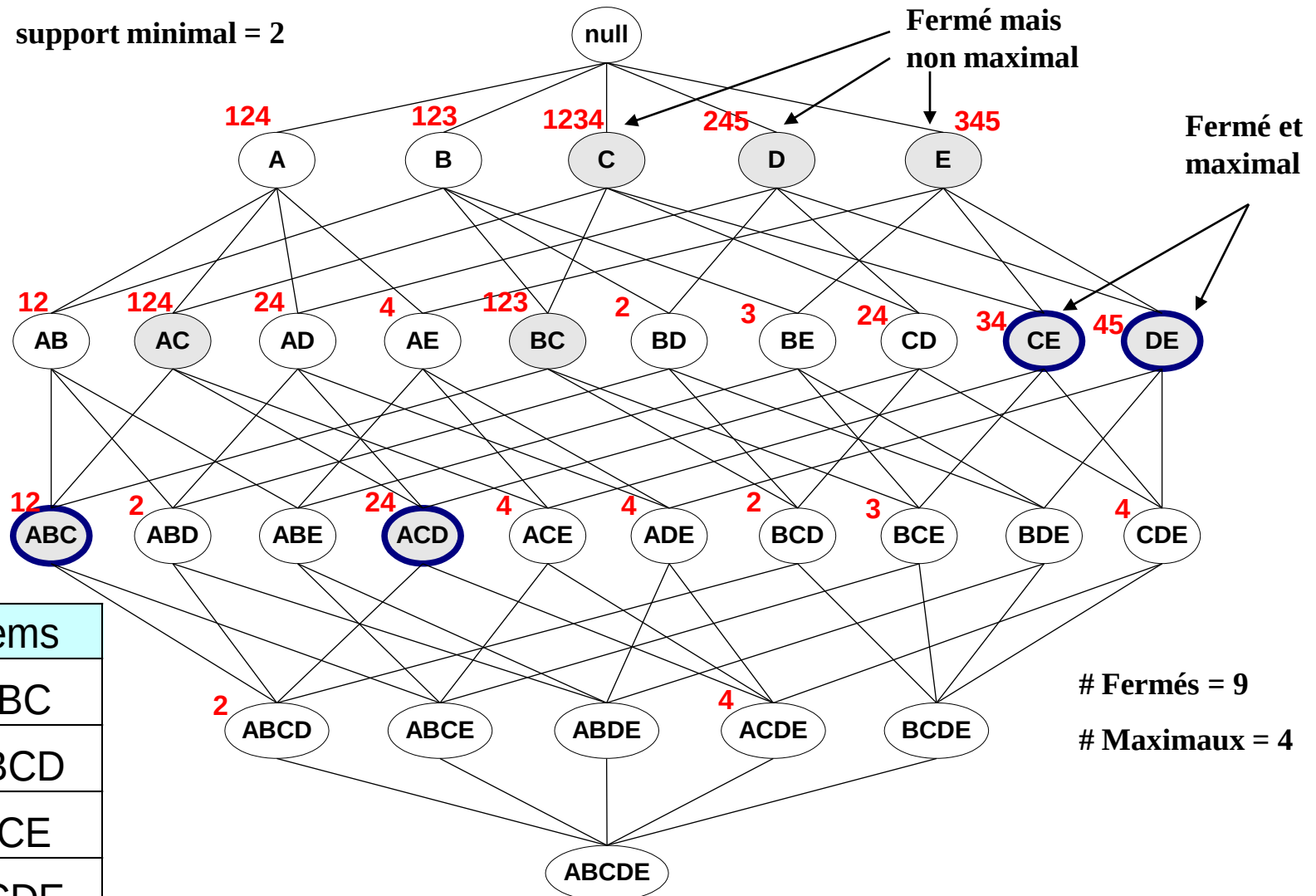
Les maximaux

Un itemset fréquent est maximal si aucun de ses sur-ensembles immédiats n'est fréquent



Maximaux / Fermés fréquents

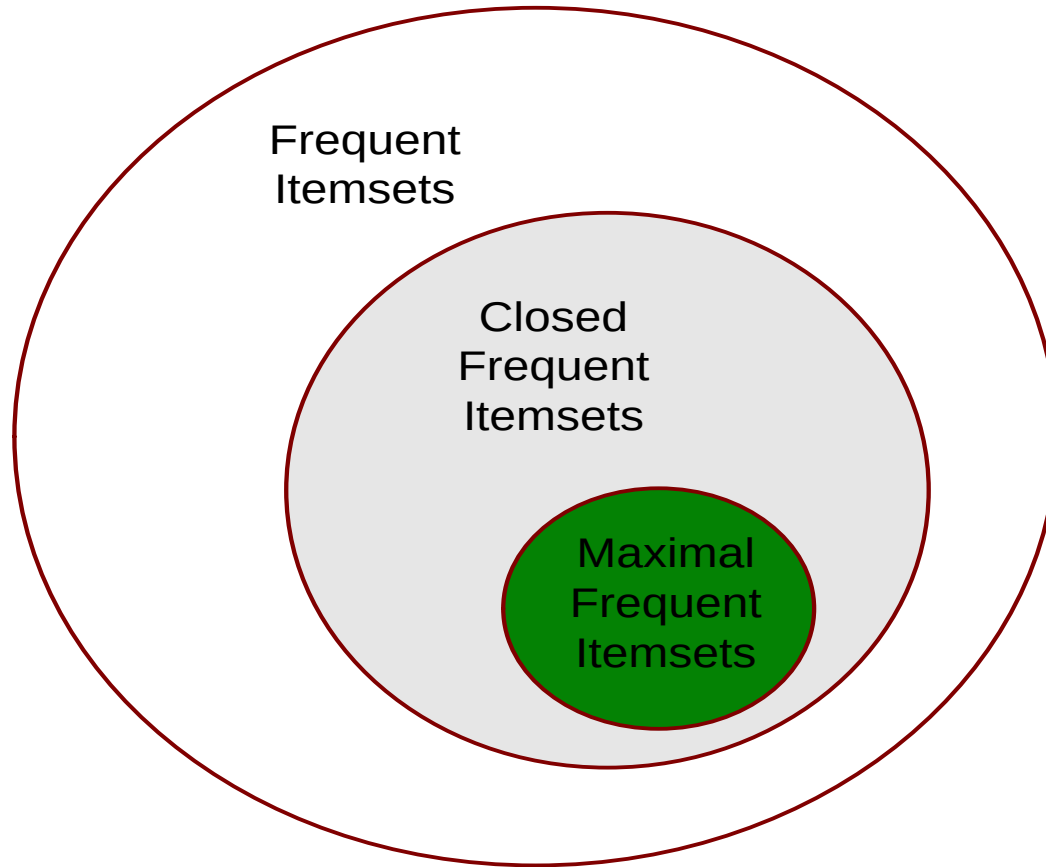
support minimal = 2



Fermés = 9

Maximaux = 4

Maximaux / Fermés fréquents



Les clés et les fermés

- Soit I l'ensemble des items, T l'ensemble des transactions et une relation définie sur $I \times T$
 - $x R y$ si l'item x appartient à la transaction y .
- Pour tout $X \subseteq I$, soit $f(X) = \{ y \in T / (\forall x \in I) (x R y) \}$
 - $f(X)$ est l'ensemble des transactions contenant X
- Pour tout $Y \subseteq T$, soit $g(Y) = \{ x \in I / (\forall y \in T) (x R y) \}$
 - $g(Y)$ est l'ensemble des items contenus dans toutes les transactions de Y
- L'application h définie pour tout $X \subseteq I$ par $h(X) = g(f(X))$ est une fermeture
 - Monotone : $X \subseteq X' \Rightarrow h(X) \subseteq h(X')$
 - Extensive : $X \subseteq h(X)$
 - Idempotente : $h(h(X)) = h(X)$

Les clés et les fermés

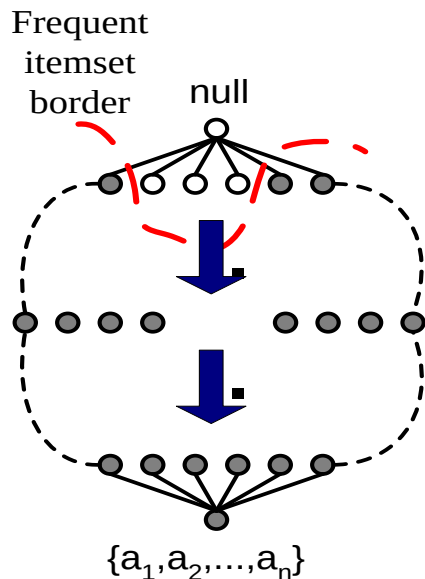
- Pour tout $X \subseteq I$, X est fermé si $X = h(X)$
- X' est un générateur de X si c'est un ensemble minimal (au sens de l'inclusion) tel que $h(X') = X$
- X' est clef si c'est le générateur d'un fermé
- Soit θ la relation définie sur $P(I) \times P(I)$ pour tout (X, Y) appartenant à $P(I) \times P(I)$ par $X \theta X'$ si $h(X) = h(X')$
 - Les fermés sont les éléments maximaux des classes d'équivalence
 - Les clefs sont les éléments minimaux des classes d'équivalence

Représentation condensée

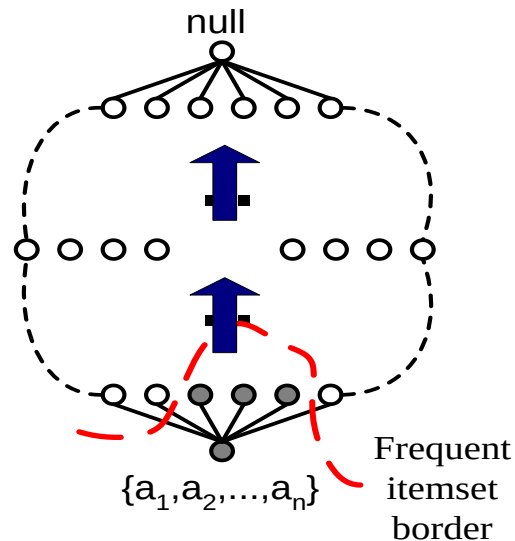
- Les clefs et les fermés
 - Pour tout $X \subseteq I$, on a $\text{sup}(X) = |f(X)| / |T|$
 - Etant donné C et G l'ensemble des fermés (closed) et clefs (generators)
 - $\text{sup}(X) = \max \{ \text{sup}(X') / X' \in C \text{ et } X \subseteq X' \}$
 - $\text{sup}(X) = \min \{ \text{sup}(X') / X' \in G \text{ et } X' \subseteq X \}$
- Notion de représentation condensée
 - Sans accéder aux données, on peut calculer les supports de tous les itemsets à partir des supports des clefs ou des fermés
 - L'algorithme Pascal permet de calculer toutes les clefs fréquentes d'un ensemble de transactions

Méthodes alternatives pour la génération des itemsets fréquents

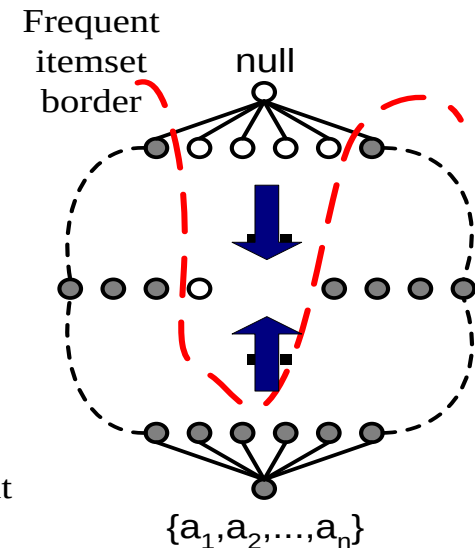
- Parcours du treillis des itemsets
 - General-to-specific vs Specific-to-general



(a) General-to-specific



(b) Specific-to-general



(c) Bidirectional

Méthodes alternatives pour la génération des itemsets fréquents

- General-to-Specific
 - Apriori
 - Efficace lorsque taille maximale itemset pas très grande
 - Meilleure configuration des itemsets (a)
- Specific-to-General
 - Pour trouver les maximaux fréquents
 - Si un maximal est trouvé, plus besoin de regarder ses sous-ensembles

Méthodes alternatives pour la génération des itemsets fréquents

- Parcours du treillis des itemsets
 - Classes d'équivalence

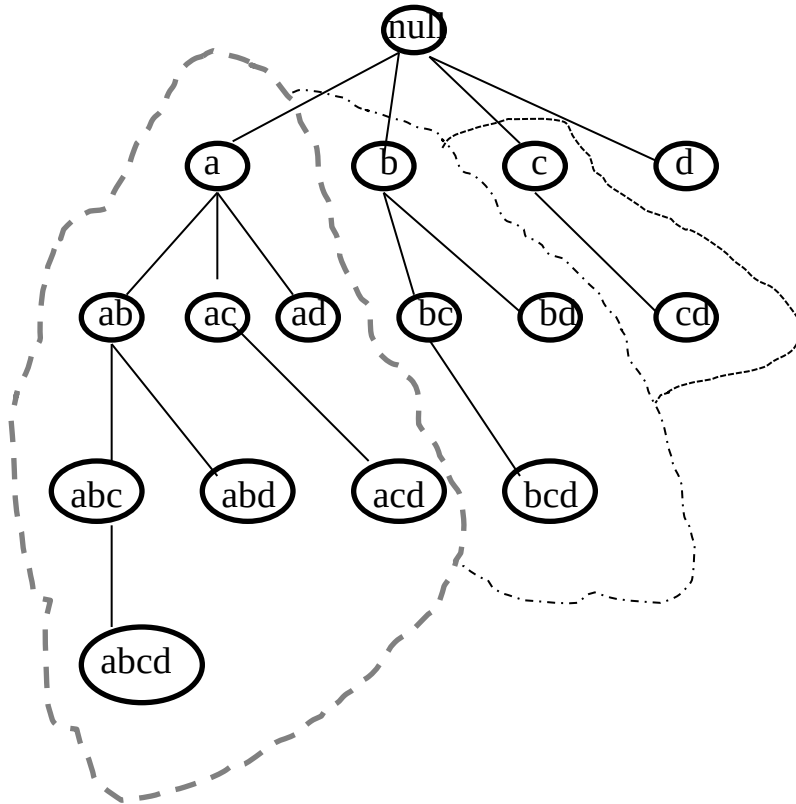
Partitionner le treillis en groupes de noeuds disjoints

Rechercher les itemsets fréquents dans une classe puis aller à une autre

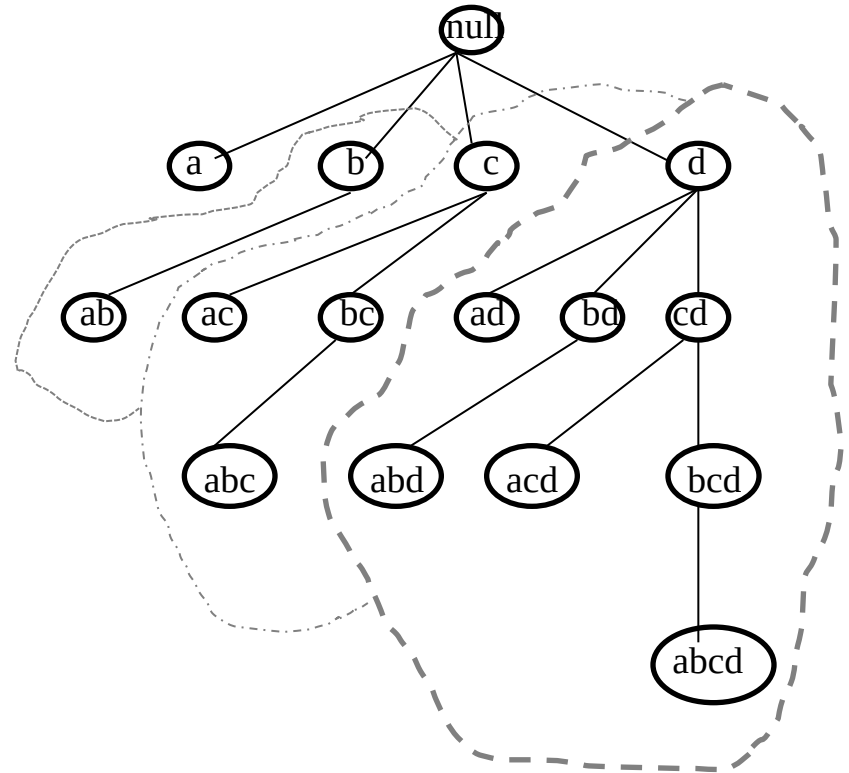
Exemples de classes d'équivalence :

- Parcours par niveau : classes selon la taille
- Classes selon le préfixe d'un itemset
- Classes selon le suffixe d'un itemset

Classes d'équivalence



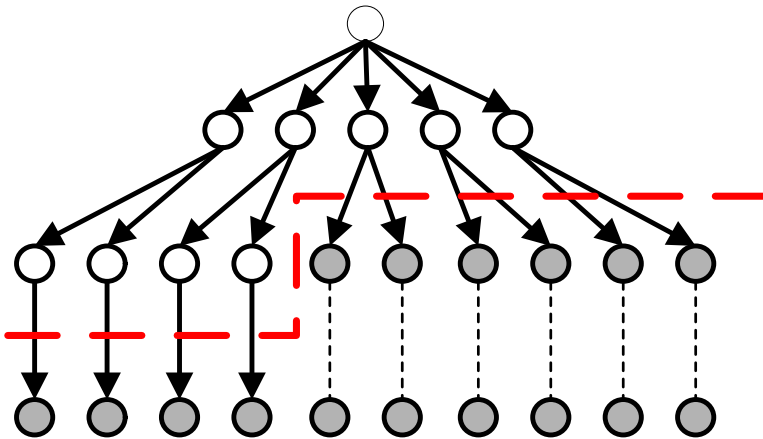
Prefix tree



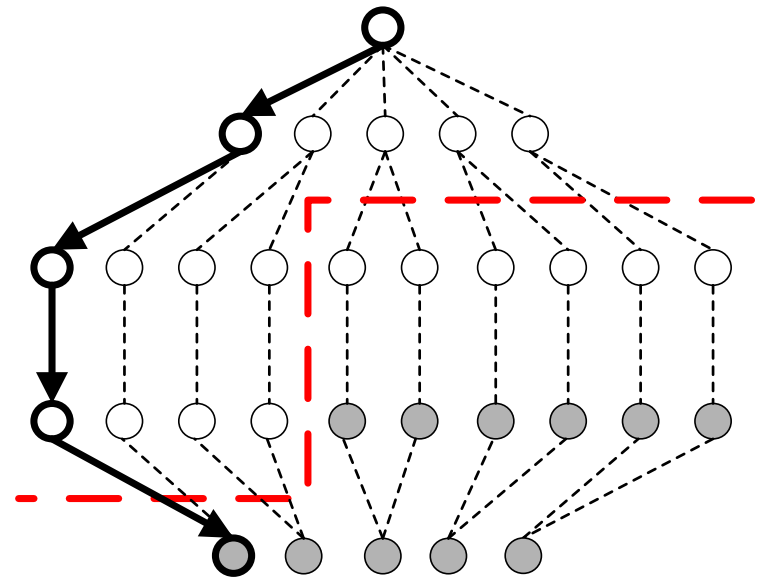
Suffix-tree

Méthodes alternatives pour la génération des itemsets fréquents

- Parcours du treillis des itemsets
 - Breadth-first vs Depth-first



(a) Breadth first



(b) Depth first

Breadth-first vs Depth-first

- Breadth-first
 - Utilisé par Apriori
- Depth-first
 - Par exemple : a(+), ab(+), abc(+), abcd(-); retourne à abce
 - Utilisé pour trouver les maximaux fréquents
 - Elagage lorsqu'un maximal fréquent est trouvé

Conclusion

- Perspectives
 - Utilisation d'autres mesures d'intérêt que le support et la confiance
 - Conviction, force d'implication,...
 - Prise en compte de contraintes anti-monotones autres que les mesures d'intérêt classiques
 - Recherche des règles d'association dont la somme des articles dépasse une certaine somme
 - Approche itérative
 - Comment utiliser les résultats de calculs antérieurs pour optimiser les calculs de nouvelles requêtes d'extraction?
 - Comment stocker efficacement les résultats de requêtes d'extraction?

Conclusion

- Approche : Résumés d'ensembles de règles
 - Problème : profusion de règles
 - Interprétation difficile



Petite taille
Non redondance
Représentativité

Conclusion

- Généralisation des règles d'association
 - Prise en compte de hiérarchies sur les attributs
 - Si {Alcool, Légumes} n'est pas intéressant, {Bière, Carottes} n'est pas intéressant
 - Prise en compte de plusieurs tables
 - Par introduction de la Programmation Logique Inductive (ILP)
 - Règles unidimensionnelles
$$\text{buys}(X, \text{"milk"}) \rightarrow \text{buys}(X, \text{"bread"})$$
 - Règles multidimensionnelles (plus de 2 prédicats)
$$\text{Client}(x, \text{"Avocat"}, \text{"Dakar"}), \text{Ventes}(x,y) \Rightarrow \text{Produit}(y, \text{"Lait"})$$
$$[15\%, 75\%]$$